

Modèles génératifs

Modèles de langue statistiques, n-grammes et génération de texte

Traduit et adapté depuis Bootstrap World (Fall 2026)

Adapté au contexte francophone — corpus de la chanson « Une vieille dame avala une
mouche »

Licence Creative Commons 4.0 International

*Ces supports ont été développés en partie grâce au soutien de la National Science Foundation
(bourses 1042210, 1535276, 1648684, 1738598, 2031479 et 1501927).*

Table des matières

1	Introduction à la modélisation statistique du langage	3
1.1	Lancement	3
1.2	Synthèse	6
2	Construire un modèle de langue statistique	7
2.1	Lancement : une chanson enfantine française	7
2.1.1	La table des unigrammes est effectivement un sac de mots	9
2.2	Exploration : construire le modèle en Pyret	10
2.2.1	Approfondissons : la probabilité conditionnelle	10
2.3	Synthèse	12
3	Échantillonner à partir du modèle	13
3.1	Lancement	13
3.2	Exploration : générer du texte	14
3.3	Synthèse	16

Introduction à la modélisation statistique du langage

Objectif de la section

Les élèves découvrent comment la prédiction de texte repose sur la modélisation statistique du langage.

Lancement

En envoyant des textos, des courriels, ou même en effectuant une recherche web, vous avez probablement remarqué que vos phrases sont parfois terminées pour vous ! Le texte prédictif est maintenant une fonctionnalité courante de nombreuses applications, et il est rendu possible par une certaine forme d'IA : le **modèle de langue**.

Considérez ce début de phrase : « Ferme la... »

Réflexion

- Quel mot pourrait venir ensuite ?
- En classe, y a-t-il plus d'un « mot suivant » possible ?
- Lequel est le plus probable ?

Notes pour l'enseignant·e

Les élèves penseront à des mots comme « porte », « boîte », « fenêtre », « radio »... Affichez les résultats de la classe. Si un mot est répété, résumez les résultats en tenant le compte de combien d'élèves proposent chaque mot particulier.

Réflexion

Regardons de plus près les mots que nous avons générés.

- Quels mots ont été proposés le plus souvent ?
- Le moins souvent ?
- Êtes-vous surpris·e par les mots les plus et les moins courants ? Pourquoi ?
- Quelle fraction de notre classe a proposé chacun de nos trois principaux mots ?
 - Par exemple : s'il y a 25 élèves dans votre classe et que 8 élèves ont choisi « porte », il a été choisi $8/25^{\text{es}}$ du temps.

Point clé

Les modèles de langue statistiques reposent sur l'hypothèse que la fraction du temps où un mot suit une série de mots dans un corpus d'entraînement peut aider à prédire la probabilité que ce soit le bon mot pour suivre cette même série de mots dans une situation future. Dans cette leçon, nous appellerons cette fraction sa **probabilité**.

Réflexion

- **Pensez-vous qu'une application de messagerie avec une fonctionnalité de texte prédictif produirait les mêmes résultats que notre classe ? Pourquoi ?**
 - Les réponses varieront. Invitez à la conversation.
 - Une application ne proposera probablement pas tous les mêmes mots.
 - Notre classe est un assez petit échantillon, tout-e-s de la même génération et de la même géographie, et nous sommes tou-te-s à l'école — nos esprits sont préparés à répondre différemment que si nous étions ailleurs.

Point clé

Comme d'autres systèmes d'IA dont nous avons parlé, les fonctionnalités de texte prédictif sont entraînées sur une grande quantité de données, en l'occurrence du texte.

Pendant l'entraînement :

1. L'ordinateur extrait chaque utilisation de la phrase « Ferme la » du corpus, ainsi que le mot qui suit.
2. Il détermine ensuite la probabilité de chaque mot suivant possible.

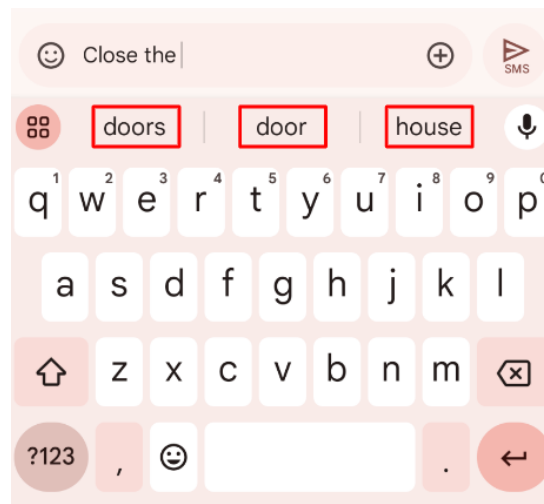


FIGURE 1 – Capture d'écran d'une application de messagerie proposant des suggestions après la phrase « Ferme la » (l'original anglais proposait « Close the »).

Point clé**Quand elle fait une suggestion :**

L'application propose les mots qui avaient la plus forte probabilité de suivre « Ferme la » dans le corpus sur lequel elle a été entraînée. Considérez cette capture d'écran d'une application de messagerie. Dans le corpus d'entraînement sur lequel elle a été construite, « porte », « portes » et « maison » apparaissaient le plus souvent après « Ferme la ». En conséquence, ces mots sont proposés.

Mais est-ce que tous·tes les utilisateur·ices reçoivent les *mêmes* complétions ?

Réflexion

Qu'en pensez-vous ? Votre téléphone proposera-t-il les mêmes mots que les téléphones de vos camarades ? Pourquoi ?

Notes pour l'enseignant·e

Avant d'entrer dans l'explication détaillée ci-dessous, invitez à la discussion ! Cette question est une excellente occasion de revisiter les concepts précédents, en particulier le processus d'entraînement et d'échantillonnage que les élèves ont abordé en parlant d'*Évaluer des arbres de décision* et de *Modèles de régression simples*.

Point clé

Pour toutes sortes de raisons, tous les téléphones ne proposeront pas les mêmes complétions :

- S'il y a plusieurs mots avec une probabilité égale, le système sélectionnera le mot *au hasard*. Par conséquent, même le même téléphone ne proposera pas nécessairement le même mot à chaque fois.
- Tous les téléphones n'utilisent pas le même corpus, les mêmes règles de prédiction, ou la même quantité d'aléatoire.
- Une application de messagerie inclut aussi un modèle de *les mots que le propriétaire utilise*, pour personnaliser les suggestions selon la façon dont l'utilisateur·ice écrit.
- Certaines fonctionnalités de texte prédictif prennent même en compte le message spécifique en cours de rédaction ! Taper la première lettre d'un mot inhabituel que vous avez utilisé dans le même message déclenche l'application pour proposer ce mot inhabituel.

Point clé

Vous venez d'examiner le fonctionnement et l'utilisation contextuelle d'un modèle de langue. Espérons que vous avez découvert que, même s'il peut parfois *sembler* que votre application de messagerie peut lire dans vos pensées... elle ne le peut pas. Elle ne connaît pas les règles de grammaire, ni la signification des mots, ni vos intentions lorsque vous composez un texto. Elle ne connaît que la probabilité, qu'elle utilise de manières souvent très impressionnantes (mais parfois pas!).

Tout au long de la leçon, nous explorerons le très important « parfois pas » entre parenthèses, ci-dessus.

Point clé

Utiliser des ordinateurs pour traiter des langues, souvent en consommant un corpus et en construisant un modèle, est appelé **traitement du langage naturel** (*Natural Language Processing*). Dans cette leçon, nous explorerons plus en détail ce que le traitement du langage naturel implique exactement.

Synthèse

Réflexion

- **La modélisation statistique du langage serait-elle possible pour d'autres langues humaines parlées? Lesquelles?**
 - La modélisation statistique du langage fonctionnera pour n'importe quelle langue! L'IA n'a pas besoin de « connaître » quoi que ce soit des règles de grammaire; elle suit juste des règles qui lui permettent d'identifier des motifs.
- **Pouvez-vous penser à d'autres choses, en dehors des langues humaines parlées, pour lesquelles une approche similaire pourrait fonctionner?**
 - Avec la modélisation statistique du langage, l'IA peut composer de la musique, jouer aux échecs, et plus encore. Le « texte » n'a pas besoin d'être composé de mots: n'importe quelle notation symbolique fera l'affaire tant qu'elle utilise des espaces pour séparer les « mots » symboliques.

Construire un modèle de langue statistique

Objectif de la section

Les élèves construisent un modèle de langue statistique en décomposant le texte et en calculant les probabilités de différents mots qui se suivent.

Lancement : une chanson enfantine française

La meilleure façon de donner du sens à la modélisation statistique du langage est de l'essayer soi-même ! Nous allons commencer par construire un modèle.

Pour notre corpus, nous utiliserons la chanson enfantine française « **Une vieille dame avala une mouche** » — l'équivalent français de la chanson anglaise *There Was an Old Lady Who Swallowed a Fly* — qui raconte l'histoire absurde d'une vieille dame qui avale une mouche, et la série d'événements malheureux qui s'ensuit.

Point clé

Le corpus

Notre fichier de démarrage (`Modeles-Generatifs-Starter.arr`) contient le corpus suivant : le titre et les paroles de la chanson.

Une vieille dame avala une mouche

Une vieille dame avala une mouche,

Je ne sais pas pourquoi elle avala une mouche – peut-être qu'elle va mourir !

Une vieille dame avala une araignée

Qui gigotait et frétilait dans son ventre ;

Elle avala l'araignée pour attraper la mouche ;

Je ne sais pas pourquoi elle avala une mouche – peut-être qu'elle va mourir !

Une vieille dame avala un oiseau ;

Quelle absurdité d'avalier un oiseau !

Elle avala l'oiseau pour attraper l'araignée

Qui gigotait et frétilait dans son ventre,

Elle avala l'araignée pour attraper la mouche ;

Je ne sais pas pourquoi elle avala une mouche – peut-être qu'elle va mourir !

Une vieille dame avala un chat ;

Eh bien, figurez-vous, elle avala un chat !

Elle avala le chat pour attraper l'oiseau,

Elle avala l'oiseau pour attraper l'araignée

Qui gigotait et frétilait dans son ventre,

Elle avala l'araignée pour attraper la mouche ;

Je ne sais pas pourquoi elle avala une mouche – peut-être qu'elle va mourir !

Comme la version anglaise, le corpus inclut le titre. La structure est identique : une mouche, une araignée, un oiseau, un chat — chaque couplet ajoutant un animal plus gros que le précédent, et le refrain répétant la même phrase.

Activité

Lisez les paroles de la chanson. Vous avez découpé la première ligne de notre corpus en morceaux de tailles différentes (un mot à la fois, deux mots à la fois, etc.) :

Taille du morceau	Quantité	Décomposition
1 mot	6	(Une) (vieille) (dame) (avala) (une) (mouche)
2 mots	5	(Une vieille) (vieille dame) (dame avala) (avala une) (une mouche)
3 mots	4	(Une vieille dame) (vieille dame avala) (dame avala une) (avala une mouche)

Point clé**Les n-grammes**

Le mot formel que les informaticien·nes utilisent dans ce contexte n'est pas « morceau » mais **n-gramme**. Dans un n -gramme, n représente le nombre de mots dans le morceau. Pour les cas particuliers où n vaut 1, 2, ou 3, les n -grammes sont appelés **unigrammes**, **bigrammes**, et **trigrammes**.

Réflexion

- Quelles autres collections de mots commençant par « uni », « bi » et « tri » avez-vous déjà rencontrées ?
 - Les réponses varieront ! *unicycle* (monocycle), *bicycle* (bicyclette), *tricycle* ; monôme, binôme, trinôme ; biceps, triceps ; triangle ; licorne...

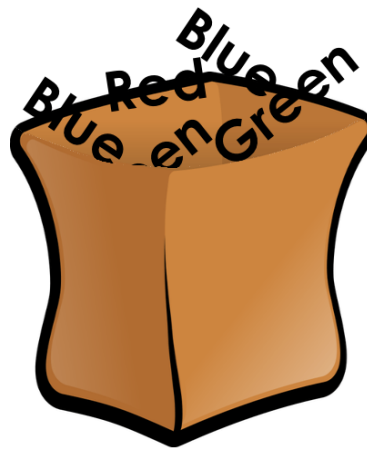
La table des unigrammes est effectivement un sac de mots

FIGURE 2 – Un sac de dessin animé, rempli avec les mots « Red », « Green » et « Blue ».

Point clé

Quand on se restreint à des mots uniques (unigrammes), un modèle n -grammes stocke exactement les mêmes données qu'un **sac de mots** (*bag of words*), même s'ils proviennent de concepts très différents :

- Le modèle sac de mots représente des documents comme des coordonnées à n dimensions, leur permettant d'être utilisés pour la classification ou la régression.
- Les unigrammes proviennent de la modélisation du langage et — avec les bigrammes et trigrammes — sont utilisés pour *calculer des probabilités*.

Les mêmes données, des objectifs très différents.

Exploration : construire le modèle en Pyret

Activité

- Ouvrez le fichier **Modèles Génératifs** (Modeles-Generatifs-Starter.arr), enregistrez une copie, et cliquez sur « Run ».
- Évaluez `corpus` dans la Zone d'Interactions.
- Évaluez `generate-ngrams(corpus, 1)`.

Approfondissons : la probabilité conditionnelle

La phrase « une vieille dame avala une... » est répétée dans notre corpus ! Concentrons-nous sur le mot « vieille » de cette phrase.

Réflexion

- **En vous référant au corpus (les paroles *et le titre*) : combien de fois le mot « vieille » apparaît-il dans la chanson ?**
 - 5.
- **Dans ce corpus, combien de fois « vieille » a-t-il été suivi de « dame » ?**
 - 5.
- **Quelle est la probabilité que « vieille » soit suivi de « dame » ?**
 - 5/5, soit 100 %.

Dans l'exemple que vous venez de traiter, vous avez calculé la **probabilité conditionnelle** que « dame » apparaisse après « vieille » en divisant le nombre de fois où « **vieille** » est suivi de « **dame** » (cinq fois) par le nombre de fois où nous voyons « **vieille** » suivi de **n'importe quoi** (cinq fois).

Cette fraction représente la probabilité que « dame » vienne après « vieille » :

$$p(\text{dame} \mid \text{vieille}) = \frac{\text{nombre de (vieille dame)}}{\text{nombre de (vieille...)}} = \frac{5}{5}$$

Activité

Complétez *Construire un modèle de langue*.

- **Quel corpus d'entraînement avons-nous utilisé pour construire le modèle de langue ?**
 - Les paroles de la chanson, y compris le titre de la chanson, étaient notre corpus.
- **Faites une prédiction : comment pouvons-nous utiliser les ratios que nous avons complétés ?**
 - Nous pouvons nous référer à nos ratios pour déterminer quel mot est le plus susceptible de suivre un mot donné.

Notes pour l'enseignant·e

Si vous voulez explorer la probabilité et l'échantillonnage plus en profondeur, consultez la leçon Bootstrap sur la probabilité et l'inférence !

Activité

- Retournez au fichier **Modèles Génératifs**.
- **Dans la Zone de Définitions**, construisez votre modèle de langue en évaluant `m = build-lang-model(corpus)`, puis cliquez sur « Run ».
- Évaluez `m` pour voir à quoi il ressemble !
- Pour trouver la probabilité qu'un mot suive un autre, évaluez :

```
next-word-probability(m, "une", "mouche")
```

Point clé

Adaptation française de la normalisation

Dans la leçon originale anglaise, la bibliothèque `ai-library.arr` de Bootstrap fournit une fonction `massage-string` qui met le texte en minuscules et supprime la ponctuation, mais qui ne conserve que les lettres `a-z/A-Z`. Cette fonction est utilisée *en interne* par toutes les fonctions `n-grammes` de la bibliothèque.

Notre fichier de démarrage ne réimplémente aucune fonction : il charge une **version française de la bibliothèque** (`ai-library-fr.arr`, hébergée dans ce dépôt), dans laquelle seule la normalisation du texte est adaptée au français. Les fonctions `n-grammes` (`generate-ngrams`, `build-lang-model`, `completions`, `next-word-probability`, `choose-completion`, `generate-from`) sont les **fonctions originales de Bootstrap**, inchangées. Les différences de la version française :

1. **Les accents sont conservés** — `était` reste `était` (l'original aurait supprimé le `é`!).
2. **Les apostrophes deviennent des espaces** — `l'araignée` devient `l araignée`, ce qui permet de traiter l'article élide `l'` comme un mot à part entière.
3. **Les tirets sont supprimés** — `peut-être` devient `peut être` dans le modèle.

Synthèse

Réflexion

Dans notre corpus de chanson,

- il y avait *quatre* complétions possibles pour l'unigramme « avala » : une, l, un, le.
- il n'y avait que *deux* complétions possibles pour le trigramme « elle avala l » : araignée et oiseau.

Nous pouvons dire que, *dans ce corpus*, à mesure que le n-gramme devient plus long, le nombre d'options de complétion diminue.

- **Pensez-vous que cela sera généralement vrai pour d'autres corpus : à mesure que le n-gramme devient plus long, le nombre d'options de complétion diminue ?**
 - Oui, en général, c'est une affirmation vraie : les phrases plus longues ont moins de complétions possibles que les mots uniques.

Échantillonner à partir du modèle

Objectif de la section

Les élèves utilisent leur modèle de langue statistique de manière générative, pour produire une sortie.

Lancement

Une fois le modèle de langue construit, que pouvons-nous en faire ? Nous pouvons l'utiliser de manière générative : nous pouvons produire une sortie !

Point clé

Comment pourrions-nous procéder ?

1. Nous pouvons commencer par choisir notre premier mot. Une approche courante est de demander : « Quel est le n -gramme le plus courant du corpus ? », mais nous pouvons aussi choisir le mot de départ nous-mêmes, si nous le voulons.
2. Ensuite, nous demandons : « Étant donné le premier n -gramme, quel est le successeur le plus courant ? »
3. Nous répétons cette deuxième étape pour toujours ! ... ou, plus réalistement, jusqu'à ce que nous décidions d'arrêter le programme. Un modèle de langue statistique simple, cependant, générera du texte *ad infinitum*.

Exploration : générer du texte

Point clé

Nous avons créé des fonctions auxiliaires pour cela, aussi :

```
# completions :: (Model model, String debut) ->
Table
# choose-completion :: (Model model, String debut,
Number n) -> String
```

- Nous pouvons utiliser ces contrats pour trouver toutes les complétions possibles d'une chaîne de départ :

```
completions(m, "une")
```

- Ou pour choisir les n prochains mots au hasard :

```
choose-completion(m, "une", 1)
choose-completion(m, "une", 2)
```

Notes pour l'enseignant·e

Notez que sur la page *Échantillonner à partir du modèle de langue*, les questions se construisent les unes sur les autres. En d'autres termes, un·e élève qui rate la question 2 aura aussi la question 3 incorrecte. Pour que la classe se déroule sans accroc, nous vous encourageons à exiger que les élèves vérifient leur travail avec un·e partenaire avant de passer d'une question à la suivante. Encourager les élèves à annoter les paroles de la chanson permettra aussi un comptage plus précis et donc des réponses plus correctes !

Activité

Exercice de génération de texte 1

1. Commençons notre génération de texte avec la phrase « elle ».
2. Déterminez quel mot est le plus susceptible de suivre « elle » et notez-le comme deuxième mot.
3. Déterminez quel mot est le plus susceptible de suivre le mot que vous venez d'écrire et notez-le comme troisième mot.
4. Utilisez la modélisation statistique du langage pour déterminer le quatrième mot.
5. Tout le monde dans votre classe aurait dû générer le *même* texte. Pourquoi pensez-vous que ce fut le résultat ?

Réponse attendue : elle avala une mouche

- elle → avala (11 occurrences, le plus fréquent) → une (7) → mouche (6).
Il n'y a jamais d'égalité : chaque étape a un mot clairement le plus probable.

Activité

Exercice de génération de texte 2

Voici une liste des unigrammes les plus courants du corpus : "avala" : 16 fois, "elle" : 15 fois, "une" : 12 fois, "mouche" : 9 fois.

1. Commençons par choisir le mot "une" (l'un des plus courants).
2. Déterminez quel mot est le plus susceptible de suivre ce mot : mouche (6 occurrences).
3. **Il y a deux mots qui ont une probabilité égale d'apparaître à la troisième place! Lesquels?**
 - je et peut-être – les deux avec 4 occurrences après mouche.
4. Utilisez la modélisation statistique du langage pour déterminer le quatrième mot de chacune des phrases ci-dessous, en utilisant chacun des deux mots que vous avez calculés pour la troisième place.

Réponses :

- une mouche je ne (car je → ne, 4 occurrences)
- une mouche peut-être qu (car peut-être → qu, 4 occurrences)
- **Pourquoi n'y a-t-il eu qu'un seul résultat pour l'exercice 1, alors que l'exercice 2 a deux résultats possibles?**
 - Parce que dans l'exercice 2, il y a eu une égalité entre les deux mots les plus probables pour la troisième place; le choix s'est fait au hasard.

Activité

- **Quelle phrase de quatre mots avez-vous générée?**
- **Tout le monde dans votre classe a-t-il obtenu la même phrase? Comment et pourquoi cela s'est-il produit?**
- **Que pensez-vous qu'il se passerait si vous commenciez avec un mot comme « zèbre », qui n'apparaît pas du tout dans le corpus original?**
 - La probabilité que « zèbre » soit suivi de chaque mot est la même : zéro. Peut-être que choose-completion choisirait au hasard?

Notes pour l'enseignant·e

Quand vous avez terminé, discutez : que se passerait-il si nous continuions à évaluer choose-completion, encore et encore?

- Il continuerait à choisir des mots!

Si vous êtes curieux·se de voir ce qui se passerait si nous continuions, évaluez generate-from(m, "une vieille")!

- **Obtenons-nous toujours la même sortie si nous utilisons toujours la même entrée?**
 - Non – en cas d'égalité, il choisit au hasard!

Synthèse

Réflexion

Un modèle est un résumé mathématique d'un jeu de données.

- **Que résume notre table de n-grammes à propos de ce jeu de données ?**
- **Qu'est-ce qu'elle laisse de côté ?**

Tous les modèles se trompent. Certains sont utiles.

- **N'importe quel modèle de langue sera-t-il parfait ? Pourquoi ou pourquoi pas ?**
- **Quand un modèle de langue est-il utile ?**

Point clé

Les critiques de ChatGPT et d'autres modèles de langue soulèvent diverses préoccupations. Considérez chacune d'elles, ci-dessous.

- **ChatGPT « invente » parfois des choses. Pourquoi cela se produit-il ? Que se passe-t-il réellement ?**
 - Quand ChatGPT produit des informations fausses ou trompeuses, ce n'est pas un bug. ChatGPT fait juste ce qu'il fait, en suivant le modèle comme il se doit.
- **ChatGPT a des biais visibles dans sa sortie textuelle. D'où viennent ces biais ?**
 - S'il y a des biais dans le corpus, il y aura probablement des biais dans la sortie !

Annexe A : Les nombres du corpus français

Les nombres ci-dessous ont été calculés sur le corpus français *après normalisation* (minuscules, apostrophes transformées en espaces, ponctuation supprimée — exactement ce que fait `massage-string`). Ils correspondent à ce que Pyret calcule.

Fait	Valeur
Mots au total	168
Unigrammes distincts	38
Unigramme le plus courant	ava la (16 fois)
2 ^e unigramme le plus courant	elle (15 fois)
3 ^e unigramme le plus courant	une (12 fois)
vieille → dame	5/5 = 100 %
elle → avala	11 occurrences (vs. va 4)
avala → une	7 occurrences (vs. l 5, un 3, le 1)
une → mouche	6 occurrences (vs. vieille 5, araignée 1)
mouche → je / peut-être	Égalité : 4 et 4

Annexe B : Du chant anglais au chant français

La chanson anglaise *There Was an Old Lady Who Swallowed a Fly* et sa version française *Une vieille dame avala une mouche* ont une structure identique :

Anglais	Français
There was an old lady who swallowed a fly	Une vieille dame avala une mouche
I don't know why she swallowed a fly – perhaps she'll die !	Je ne sais pas pourquoi elle avala une mouche – peut-être qu'elle va mourir !
She swallowed the spider to catch the fly	Elle avala l'araignée pour attraper la mouche
spider, bird, cat	araignée, oiseau, chat
wriggled and jiggled and tickled inside her	gigotait et frétillait dans son ventre

- Le titre est inclus dans le corpus, comme dans l'original.
- Le passé simple (avala) est le temps naturel en français pour raconter une histoire : il correspond au *simple past* anglais (swallowed).
- L'imparfait (gigotait, frétillait) est utilisé pour la description de fond — ce que faisait l'araignée dans son ventre — exactement comme l'anglais utilise le *past continuous* (that wriggled and jiggled).

- La chanson française ajoute un défi intéressant pour le modèle de langue : les **articles élidés**. En anglais, *the* est toujours un mot séparé. En français, *l'* est collé au nom suivant (*l'araignée*). Notre `message-string` transforme l'apostrophe en espace, ce qui permet de traiter *l* comme un mot à part entière — exactement comme *the* en anglais.
- Le tiret de `peut-être` est supprimé par la normalisation, ce qui donne le mot `peutetre` dans le modèle.

Annexe C : Fichiers du projet

Fichier	Description
<code>Modeles-Generatifs-Starter.arr</code>	Fichier de démarrage Pyret : charge la version française de la bibliothèque (<code>ai-library-fr.arr</code>) et définit le corpus français.
<code>libraries/ai-library-fr.arr</code>	Copie française de la bibliothèque Bootstrap : seule la normalisation du texte diffère (accents conservés, apostrophes → espaces).
<code>modeles-generatifs.tex</code> <code>images/</code>	Document de cours (ce fichier) 4 images : capture d'application de messagerie, sac de mots, plan 2D, système de coordonnées 3D

À noter : la bibliothèque `ai-library.arr` de Bootstrap fournit des fonctions `n-grammes` (`generate-ngrams`, `build-lang-model`, `completions`, `choose-completion`, `generate-from`, `next-word-probability`) qui utilisent en interne une fonction `message-string` conçue pour l'anglais (elle supprime tous les caractères en dehors de `a-z`). Ces fonctions fonctionneraient mal avec du texte français accentué. Notre fichier de démarrage charge donc une **copie française** de la bibliothèque (`ai-library-fr.arr`, hébergée dans `libraries/`) dans laquelle `message-string` est adaptée au français (conservation des accents, apostrophes transformées en espaces, tirets supprimés). Les fonctions `n-grammes` utilisées restent celles de Bootstrap, inchangées.

À propos de ces supports

Ces supports ont été développés en partie grâce au soutien de la *National Science Foundation* (bourses 1042210, 1535276, 1648684, 1738598, 2031479 et 1501927).

Bootstrap par la Communauté Bootstrap est sous licence Creative Commons 4.0 Unported.

Cette licence n'accorde pas la permission d'organiser des formations ou du développement professionnel.

Traduction et adaptation française réalisées en juillet 2026.

Source originale :

<https://www.bootstrapworld.org/materials/fall2026/en-us/lessons/stat-lang-models-generating-text/>